

Отчёт по лабораторной работе №2

Системы контроля версий. Git. Ветки. Теги. Игнорирование файлов

Студент: Чжу Ино

Группа: НПИбд-03-25

Студенческий билет: 1132258331

1. Цель работы

Изучить продвинутые возможности Git: работу с ветками, создание тегов, игнорирование файлов, просмотр истории.

2. Выполнение работы

2.1. Создание и работа с ветками

Рисунок 1: Создание ветки и переключение

```
(base) [cedar@cedar-95 ~]$ cd ~
mkdir lab14
cd lab14
git init
mkdir: cannot create directory 'lab14': File exists
Reinitialized existing Git repository in /home/cedar/lab14/.git/
```

2.2. Добавление файлов и коммит

Рисунок 2: Добавление файлов в ветке

```
(base) [cedar@cedar-95 lab14]$ git config --global core.pager cat
(base) [cedar@cedar-95 lab14]$ git config --global --list
user.name=cedar-95
user.email=1132258331@rudn.ru
core.pager=cat
```

2.3. Слияние веток и удаление

Рисунок 3: Слияние веток и удаление

```
(base) [cedar@cedar-95 lab14]$ git push
Username for 'https://github.com': git remote -v
Password for 'https://git%20remote%20-v@github.com':
```

2.4. Просмотр истории и отправка на удалённый репозиторий

Рисунок 4: Просмотр истории и push

```
(base) [cedar@cedar-95 lab14]$ git push
Username for 'https://github.com': cedar-95
Password for 'https://cedar-95@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 668 bytes | 668.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/cedar-95/lab14.git
    7df184a..c10e59e  main -> main
(base) [cedar@cedar-95 lab14]$ git remote -v
origin  https://github.com/cedar-95/lab14.git (fetch)
origin  https://github.com/cedar-95/lab14.git (push)
```

3. Выводы

В ходе выполнения лабораторной работы №2 были изучены продвинутые возможности Git: создание и слияние веток, удаление веток, создание тегов, игнорирование файлов, просмотр истории коммитов и работа с удалённым репозиторием.

4. Ответы на контрольные вопросы

Вопрос 1: Что такое ветка в Git?

Ветка — это подвижный указатель на один из коммитов. Она позволяет вести независимую линию разработки.

Вопрос 2: Зачем нужны ветки?

- Для параллельной разработки новых функций
- Для исправления ошибок без влияния на основную версию
- Для экспериментов

Вопрос 3: Как создать новую ветку?

```
git branch <имя_ветки>
git checkout -b <имя_ветки>
Вопрос 4: Как объединить ветки?
```

```
bash
git checkout <целевая_ветка>
git merge <исходная_ветка>
Вопрос 5: Что такое тег в Git?
```

Тег — это метка, указывающая на конкретный коммит. Используется для отметки

Вопрос 6: Как создать и удалить тег?

```
bash
git tag -a v1.0.0 -m "сообщение"
git tag -d v1.0.0
git push --tags
Вопрос 7: Как игнорировать файлы?
```

Создать файл .gitignore в корне репозитория и перечислить в нём шаблоны имён

Вопрос 8: Примеры шаблонов для .gitignore

```
text
*.log
*.tmp
.idea/
__pycache__/
.DS_Store
Вопрос 9: Как посмотреть историю коммитов?
```

```
bash
git log
git log --oneline
git log --graph
git log --all
Вопрос 10: Что такое конфликт при слиянии и как его разрешить?
```

Конфликт возникает, когда две ветки изменяют одни и те же строки. Разрешение

Открыть конфликтный файл

Найти маркеры <<<<<<, =====, >>>>>>

Выбрать нужные изменения

Удалить маркеры

Выполнить git add и git commit